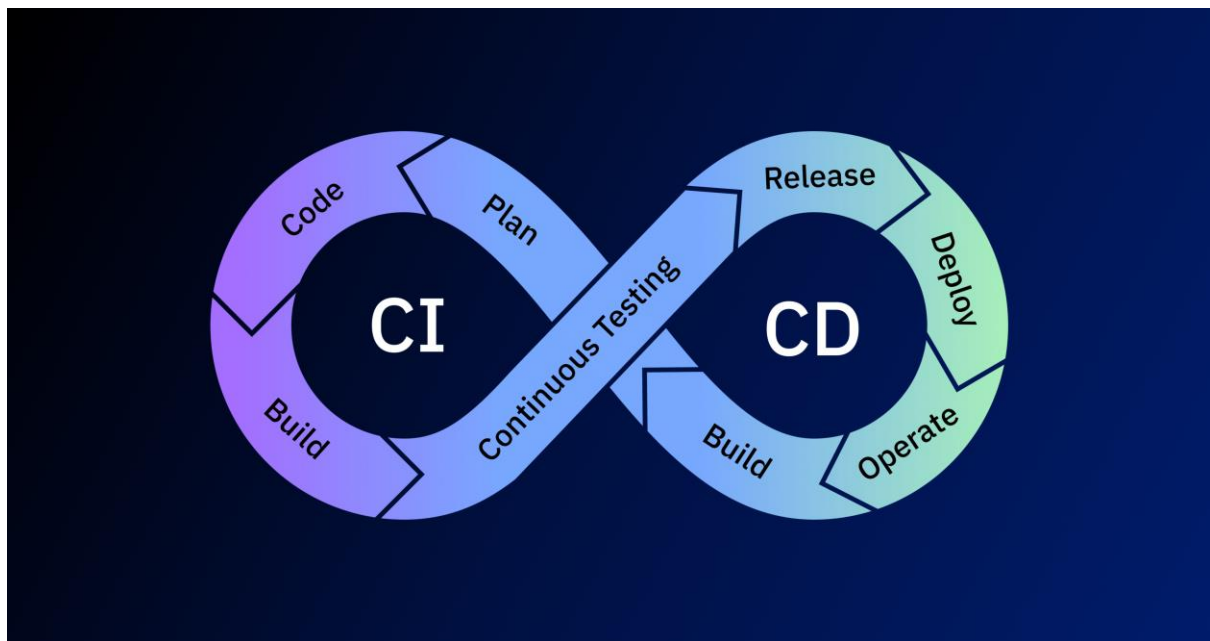




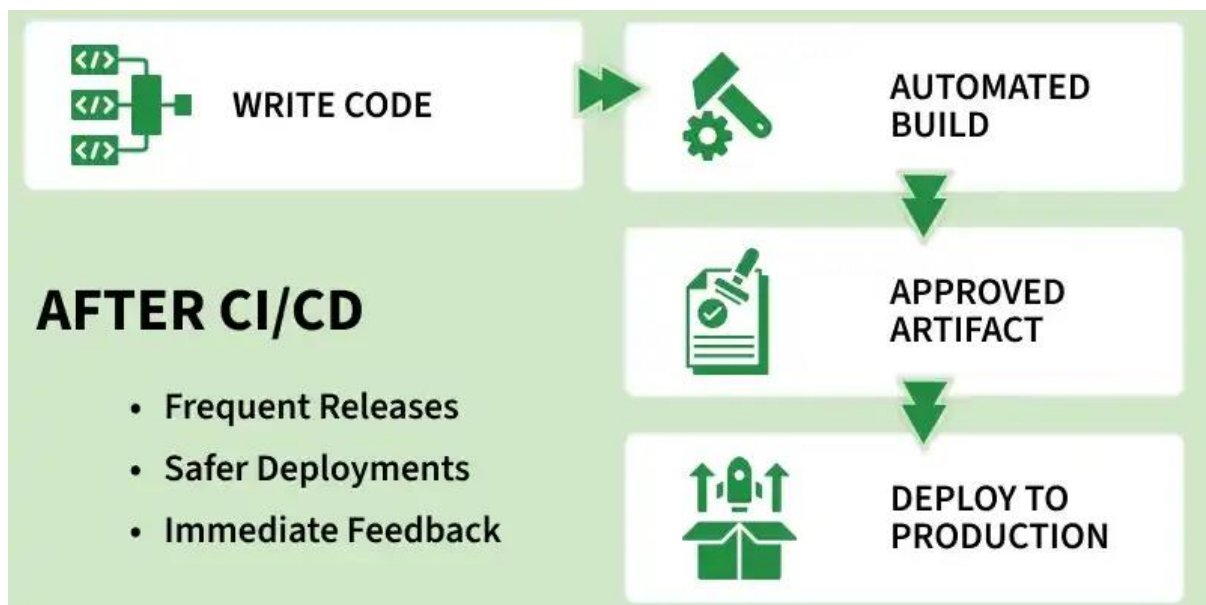
BY,
J STEVE MANUEL
Hexaware GET Trainee
Data Engineering

CI/CD Pipelines

A CI/CD pipeline is an automated workflow that enables development teams to frequently and reliably build, test, and deploy software changes. It streamlines the software development life cycle (SDLC) by automating the integration of code changes (CI) and the delivery/deployment of those changes (CD) to production or staging environments.

Core Components of a CI/CD Pipeline

- **Source:** Triggered by code changes (e.g., git push), typically using version control systems like GitHub or GitLab.
- **Build:** The code is compiled and packaged into a deployable artifact (e.g., Docker image).
- **Test:** Automated tests (unit, integration, UI) run to ensure code quality.
- **Deploy:** The validated artifact is deployed to staging or production environments.
- **Monitor:** Ongoing tracking of application performance and security, often with feedback loops.



Three Pillars of CI/CD

To effectively implement CI/CD, it is important to understand the differences between its three core concepts: Continuous Integration, Continuous Delivery, and Continuous Deployment.

1. Continuous Integration (CI)

Continuous Integration focuses on integrating code changes frequently to avoid conflicts and ensure code stability.

- **Goal:** Prevent “integration hell” caused by late code merging.
- **Process:** Developers merge code changes into the main branch frequently (often daily).
- **Automation:** Each commit triggers an automated build and unit tests.
- **Outcome:** If tests fail, the build is rejected and developers are notified immediately.

2. Continuous Delivery (CD)

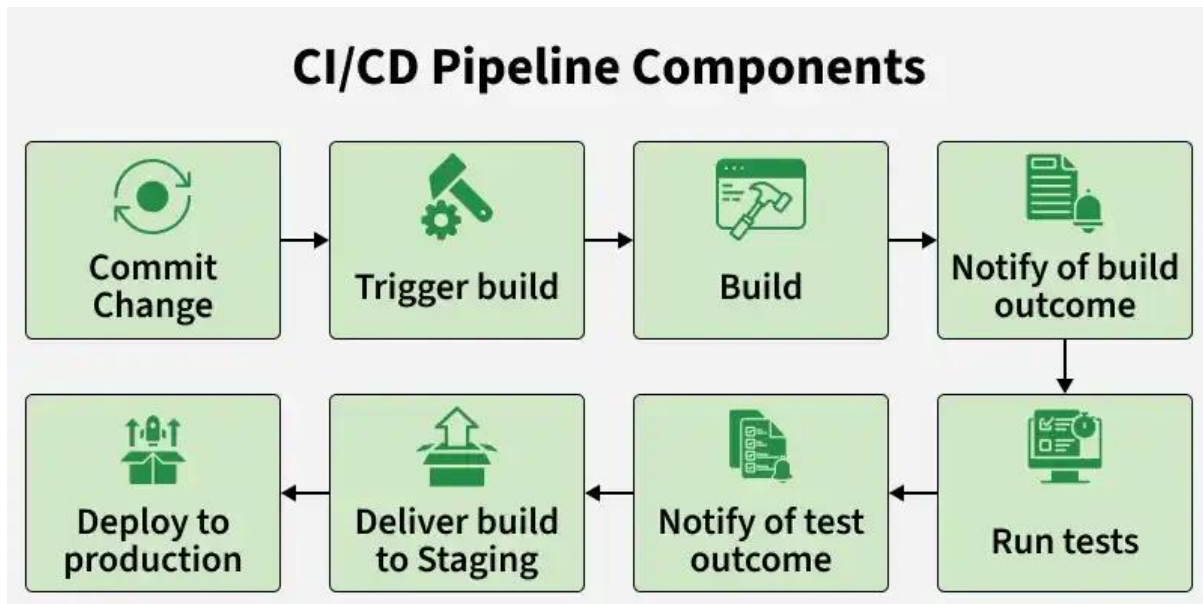
Continuous Delivery ensures that the application is always ready for release, with minimal manual effort.

- **Goal:** Keep the codebase in a release-ready state at all times.
- **Process:** After CI passes, code is deployed to a staging or testing environment.
- **Automation:** Integration, system, and performance tests are executed automatically.
- **Release:** Deployment to production is **manual**, typically triggered when needed.

3. Continuous Deployment (CD)

Continuous Deployment takes automation a step further by removing manual intervention in releases.

- **Goal:** Enable fully automated and faster production releases.
- **Process:** After all tests pass, code is automatically deployed to production.
- **Automation:** End-to-end pipeline runs without human involvement.
- **Requirement:** Requires highly reliable and comprehensive automated testing.



How do Applications / Products go live?

Going live, or deploying an application, is the final stage of the software development lifecycle where a product is moved from a development environment to a live production environment, making it accessible to users. This process involves a series of technical, compliance, and marketing steps to ensure stability and visibility.

1. Pre-Launch Preparation & Testing

Before going live, the application must be thoroughly vetted to ensure stability.

- **Final Testing:** This includes User Acceptance Testing (UAT), security audits, and performance testing to ensure the app handles high traffic.
- **Bug Fixing:** Any critical issues found during testing are resolved.
- **Final Build Creation:** The final, signed, and optimized version of the app is compiled for production (e.g., AAB for Android, IPA for iOS).

2. Setting Up the Production Environment

The application needs a place to live.

- **Backend Deployment:** APIs, databases, and servers are configured in the live cloud environment (e.g., AWS, Google Cloud).
- **Infrastructure Setup:** Hosting, scaling rules, and security certificates (SSL/HTTPS) are finalized

3. Submitting to App Stores (Mobile Apps)

For mobile applications, the "go-live" moment involves submitting the app to platforms like the Google Play Store or Apple App Store.

- **Store Listing Setup:** Preparing app names, descriptions, icons, and screenshots.
- **Compliance & Content Rating:** Filling out data safety, privacy policy, and age rating forms.
- **Mandatory Testing (Android):** New developer accounts created after November 2023 must have 20 testers test the app for 14 days before submitting.
- **App Review:** Apple and Google review the app for policy compliance, which can take several days.

4. Deploying (Web/SaaS Applications)

Web apps often use automated pipelines for deployment.

- **Continuous Deployment (CD):** Developers use CI/CD tools to automatically deploy code to production when it passes testing.
- **Feature Flags:** A technique used to enable or disable features in the live product instantly without deploying new code.

5. Post-Launch Monitoring

Going live is not the end of the process; the application must be monitored

- **Monitoring & Analytics:** Tools like Google Analytics or Crashlytics are used to track user activity and detect errors, crashes, or performance bottlenecks.
- **Maintenance:** Fixing bugs that appear after launch and updating the application with new features based on user feedback.

Different Environments while working on a Project

In software development, project environments are distinct, isolated systems where code is built, tested, and deployed to ensure stability before reaching real users. The typical progression involves four main stages:

- **Development Environment (Dev):** The local environment on a developer's machine where code is written, coded, and initially tested. It is often considered messy and unstable, designed for rapid iteration.
- **Testing/QA Environment (Test/QA):** An environment designed for testers to identify and fix issues (bugs, security vulnerabilities). It is more stable than development and allows for Quality Assurance (QA) engineers to perform regression testing.
- **Staging Environment (Stg):** A "pre-production" environment that is a mirror image of the production environment, used to test functionality and performance under real-world conditions (using dummy data that mimics production data).
- **Production Environment (Prod):** The live system where the application is accessed by real users. This environment must be stable, robust, and secured.

Key Differences Between Environments

- **Purpose:** Dev is for coding, QA for testing, Staging for validation, and Prod for operation.
- **Stability:** Dev is unstable; Prod is highly stable.
- **Data:** Test environments often use simulated data, while production handles real user data.

Other Types of Environments

- **Integration Environment:** Where developers merge their individual features into a shared, unified codebase.
- **Preview Environment:** Often created for specific feature branches to preview changes before merging.